

Department of Information Engineering Technology The
Superior University, Lahore

Mobile Application Development Lab

Experiment No.11

Flutter: API integration

Prepared for

Mam Eisha Nawaz

By:

Name: Muhammad Raza Shah

ID: SU91-BIETM-F23-026

Section: A

Semester: 6th

Total Marks: _____

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.11

Flutter: API integration

(Rubrics)

Name: Muhammad Raza Shah

Roll No: SU91-BIETM-F23-026

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		6	Sub Total marks Obtained in Psychomotor(P)					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		4	Sub Total marks Obtained in Affective (A)					

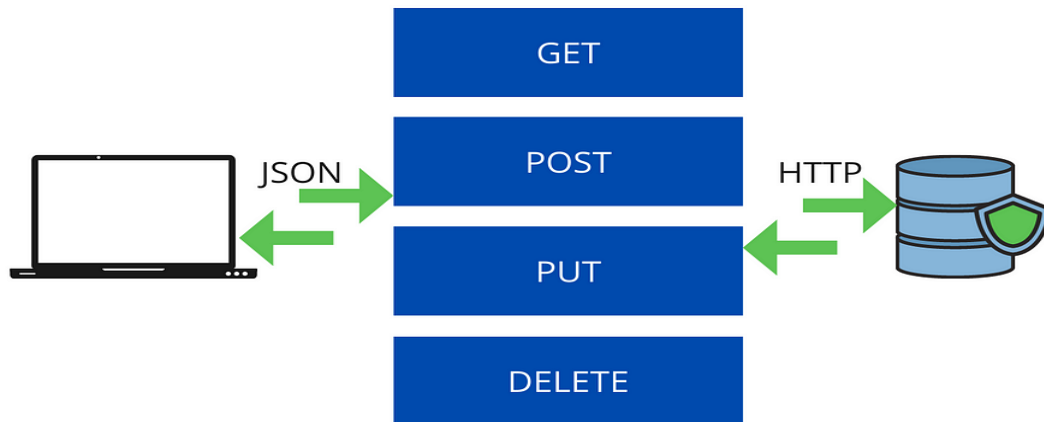
Instructor Name: _____

Total Marks (P+A): _____

Instructor Signature: _____

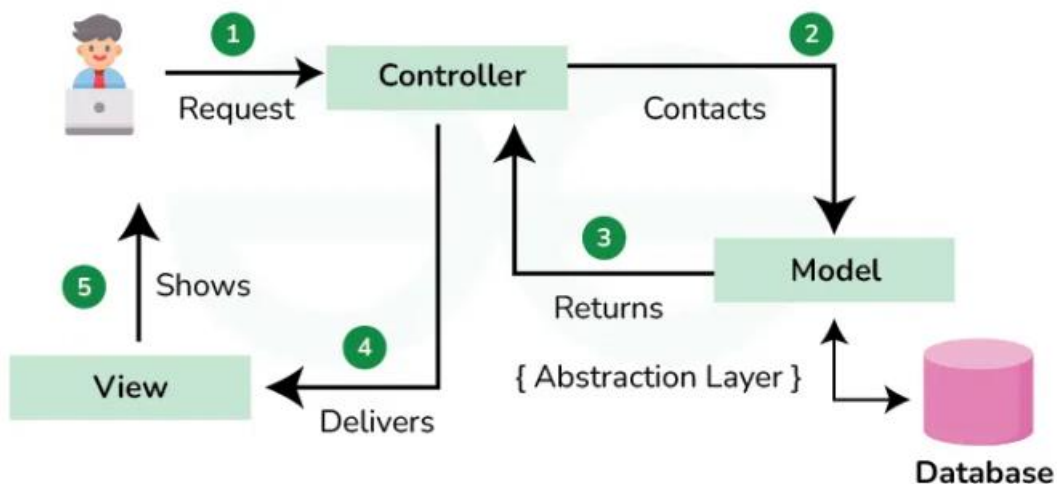
Obtained Marks (P+A): _____

11.1 How do APIs Work?



A user application can contact any database through HTTP requests. So, we can create GET, POST, PUT, or DELETE HTTP requests to a database. In return, the database sends us data or results or responses in form of JSON or HTML or XML. The JSON format is widely used. We then parse the JSON into a proper model class and use it in our app.

11.2 Integrate APIs into Flutter App



To integrate an API, we have a few steps that we can follow for our ease:

Step 1: Get API URL and endpoints.

Step 2: Add relevant packages into the app (http, dio, chopper, etc.).

Step 3: Create a constant file that stores URLs and endpoints.

Step 4: Create a model class to parse the JSON.

Step 5: Create a file that handles the API call and write specific methods to fetch data and parse it.

Step 6: Use the data in your app.

We will use [JSONPlaceholder](#) API which provides fake APIs for us to practice.

Step 1: Get API URL and endpoints.

To get the API URL (referred to as base URL) and endpoints, go to the [JSONPlaceholder](#) website and you will find different APIs that they provide. We would today use the users API that they have. Hence,

Base URL: `https://jsonplaceholder.typicode.com`

API Endpoint: `/users`

When you have a database or API provider, the base URL for all APIs remain the same however the endpoint changes depending on the API. In many API Providers, you will have to obtain a private key (API Key or Access Key) either by simply creating an account or it can be a paid one. That API Key will be just appended to the base URL. So the new base URL will be:

New Base URL = `BaseUrl/apiKey`

Example: `https://jsonplaceholder.typicode.com/abcdefghijklmnopqrstuvwxyz`

Step 2: Add relevant packages into the app (http, dio, chopper, etc.)

There are many packages available in [pub.dev](#) that we can use to integrate APIs in Flutter. The most widely used packages are:

- [http](#)
- [dio](#)

- chopper

And many more... However, http is the most basic and easy to use. The other packages are mostly the wrapper of the http package and provide additional functionalities.

Now, once you have created a new flutter project, go to pubspec.yaml file and add the http package into it. Your file should look something like the following:

```
1  name: rest_api_example
2  description: A new Flutter project.
3  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
4  version: 1.0.0+1
5
6  environment:
7    sdk: ">=2.15.1 <3.0.0"
8
9  dependencies:
10   cupertino_icons: ^1.0.2
11   flutter:
12     sdk: flutter
13   http: ^0.13.4
14
15  dev_dependencies:
16   flutter_lints: ^1.0.0
17   flutter_test:
18     sdk: flutter
19
20  flutter:
21   uses-material-design: true
```

Step 3: Create a constant file that stores URLs and endpoints.

Now, it's time to create a simple file named `constants.dart` that will hold all your URLs and endpoints. In our case, we only have one endpoint, but it's a good practice to have a separate file. Your file will look something like the following:

```
1  class ApiConstants {
2    static String baseUrl = 'https://jsonplaceholder.typicode.com';
3    static String usersEndpoint = '/users';
4  }
```

Here, we have created a class called ApiConstants and created 2 static variables so that we can access them without creating an instance of the class like `ApiConstants.baseUrl` .

Step 4: Create a model class to parse the JSON.

One way to access the data is directly using the key. However, it's a good and easy approach to create a model class, parse the JSON and get an object out of that JSON response.

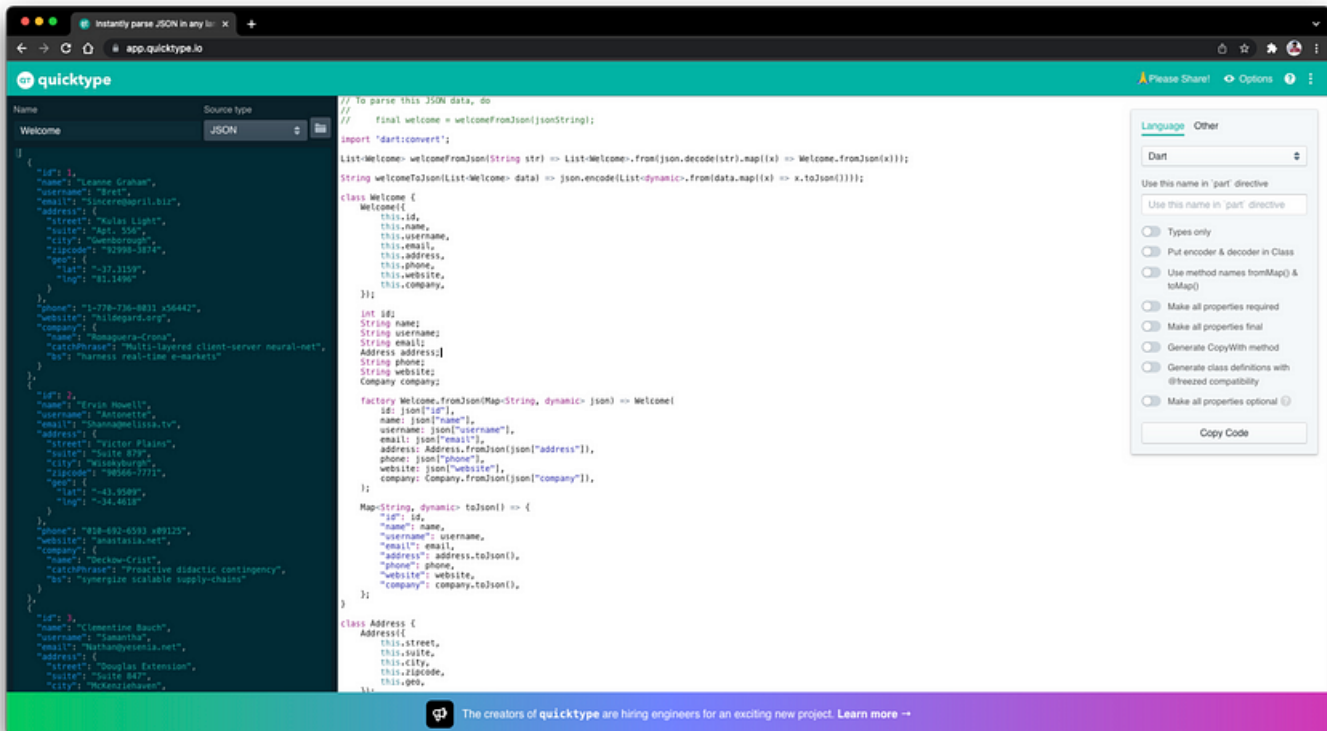
Access data directly:

```
final data = json[0]['id'];
```

In this, we tried to access the `id` of 0th record of the JSON response. This is easy only when you have limited records and entries.

Now, to create a model class, first, you will need the entire JSON response that you are getting from the API. To get that, just go to <https://jsonplaceholder.typicode.com/users> and you will be able to obtain the entire JSON response as this is a public API. Now, in the case of private API or custom backend APIs, either it will be available in the documentation or you can use [Postman](#) to hit the API and get the response.

Now, once you have the response, you can create the model class using a simple method. Go to <https://app.quicktype.io> and on the left side paste the JSON response. In the options on the right side, select `Dart` and your model class will be generated! You can then change the name of the class.



Your model class file will look something like the following:

```

1  import 'dart:convert';
2
3  List<UserModel> userModelFromJson(String str) =>
4      List<UserModel>.from(json.decode(str).map((x) => UserModel.fromJson(x)));
5
6  String userModelToJson(List<UserModel> data) =>
7      json.encode(List<dynamic>.from(data.map((x) => x.toJson())));
8
9  class UserModel {
10      UserModel({
11          required this.id,
12          required this.name,
13          required this.username,
14          required this.email,
15          required this.address,
16          required this.phone,
17          required this.website,
18          required this.company,
19      });
20
21      int id;

```

```

22     String name;
23     String username;
24     String email;
25     Address address;
26     String phone;
27     String website;
28     Company company;
29
30     factory UserModel.fromJson(Map<String, dynamic> json) => UserModel(
31         id: json["id"],
32         name: json["name"],
33         username: json["username"],
34         email: json["email"],
35         address: Address.fromJson(json["address"]),
36         phone: json["phone"],
37         website: json["website"],
38         company: Company.fromJson(json["company"]),
39     );
40
41     Map<String, dynamic> toJson() => {
42         "id": id,
43         "name": name,
44         "username": username,
45         "email": email,
46         "address": address.toJson(),
47         "phone": phone,
48         "website": website,
49         "company": company.toJson(),
50     };
51 }
52
53 class Address {
54     Address({
55         required this.street,
56         required this.suite,
57         required this.city,
58         required this.zipcode,
59         required this.geo,
60     });
61
62     String street;
63     String suite;
64     String city;
65     String zipcode;
66     Geo geo;
67
68     factory Address.fromJson(Map<String, dynamic> json) => Address(
69         street: json["street"],
70         suite: json["suite"],
71         city: json["city"],
72         zipcode: json["zipcode"],
73         geo: Geo.fromJson(json["geo"]),
74     );
75
76     Map<String, dynamic> toJson() => {
77         "street": street,
78         "suite": suite,
79         "city": city,

```



```
80         "zipcode": zipcode,
81         "geo": geo.toJson(),
82     };
83 }
84
85 class Geo {
86     Geo({
87         required this.lat,
88         required this.lng,
89     });
90
91     String lat;
92     String lng;
93
94     factory Geo.fromJson(Map<String, dynamic> json) => Geo(
95         lat: json["lat"],
96         lng: json["lng"],
97     );
98
99     Map<String, dynamic> toJson() => {
100         "lat": lat,
101         "lng": lng,
102     };
103 }
104
105 class Company {
106     Company({
107         required this.name,
108         required this.catchPhrase,
109
110         required this.bs,
111     });
112
113     String name;
114     String catchPhrase;
115     String bs;
116
117     factory Company.fromJson(Map<String, dynamic> json) => Company(
118         name: json["name"],
119         catchPhrase: json["catchPhrase"],
120         bs: json["bs"],
121     );
122
123     Map<String, dynamic> toJson() => {
124         "name": name,
125         "catchPhrase": catchPhrase,
126         "bs": bs,
127     };
128 }
```

Step 5: Create a file that handles the API call and write specific methods to fetch data and parse it.

Now, create a file called `api_service.dart` that will handle the API calls.

```
1  import 'dart:developer';
2
3  import 'package:http/http.dart' as http;
4  import 'package:rest_api_example/constants.dart';
5  import 'package:rest_api_example/model/user_model.dart';
6
7  class ApiService {
8    Future<List<UserModel>>> getUsers() async {
9      try {
10        var url = Uri.parse(ApiConstants.baseUrl + ApiConstants.usersEndpoint);
11        var response = await http.get(url);
12        if (response.statusCode == 200) {
13          List<UserModel> _model = userModelFromJson(response.body);
14          return _model;
15        }
16      } catch (e) {
17        log(e.toString());
18      }
19    }
20  }
```

8

Now, in the above file, we have created a function called `getUsers` that returns `List<UserModel>`. The first step is to hit the GET HTTP request. The next step is to check, whether the API call was successful or not. That can be checked using `response.statusCode`. If the API call will be successful, `statusCode` will be 200. If the `statusCode` is 200, we then convert the JSON (`response.body`) into a `List<UserModel>` using the method `userModelFromJson` which is available in the model class that we created and then return the model object.

Step 6: Use the data in your app.

We have created all the required files and methods for the API, which we can call our app backend. Now it's time to load this data on our UI. We won't be doing much, just a method call and load that result in UI.

```

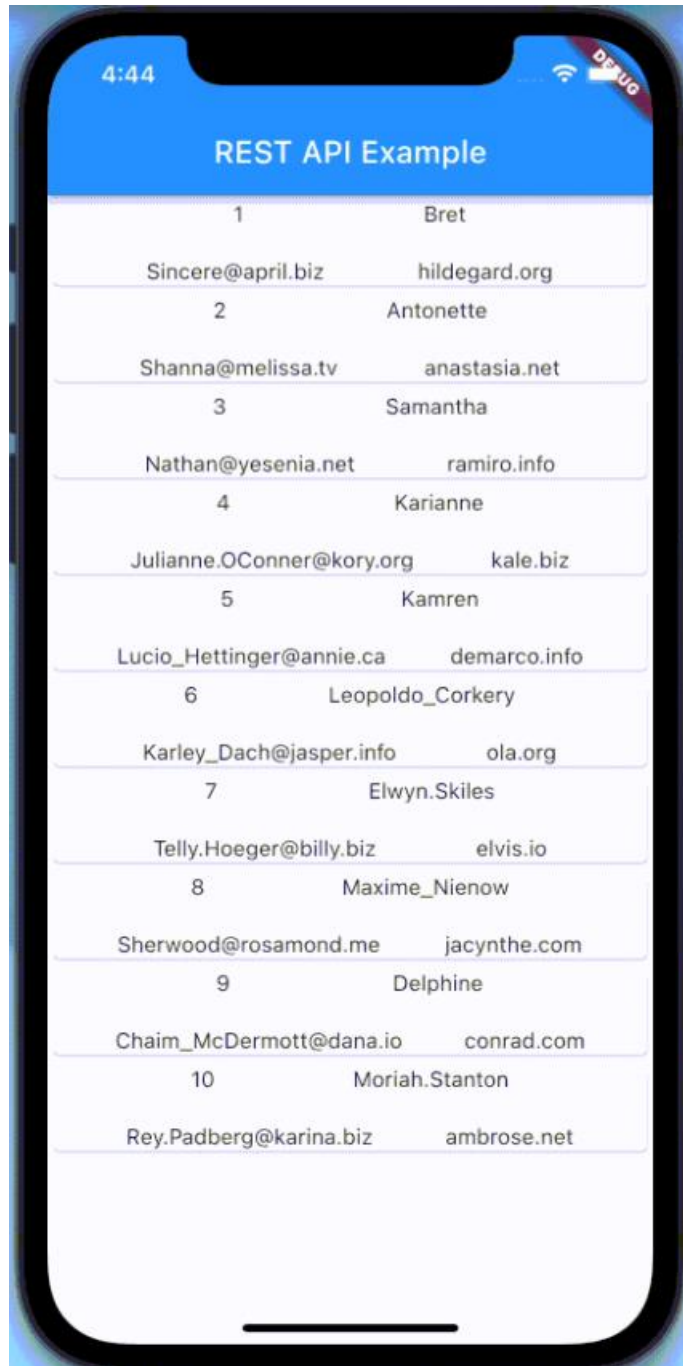
1  import 'package:flutter/material.dart';
2  import 'package:rest_api_example/model/user_model.dart';
3  import 'package:rest_api_example/services/api_service.dart';
4
5  class Home extends StatefulWidget {
6    const Home({Key? key}) : super(key: key);
7
8    @override
9    _HomeState createState() => _HomeState();
10 }
11
12 class _HomeState extends State<Home> {
13   late List<UserModel> _userModel = [];
14   @override
15   void initState() {
16     super.initState();
17     _getData();
18   }
19
20   void _getData() async {
21     _userModel = (await ApiService().getUsers())!;
22     Future.delayed(const Duration(seconds: 1)).then((value) => setState(() {}));
23   }
24
25   @override
26   Widget build(BuildContext context) {
27     return Scaffold(
28       appBar: AppBar(
29         title: const Text('REST API Example'),
30       ),
31       body: _userModel == null || _userModel!.isEmpty
32         ? const Center(
33           child: CircularProgressIndicator(),
34         )
35         : ListView.builder(
36           itemCount: _userModel!.length,
37           itemBuilder: (context, index) {
38             return Card(
39               child: Column(
40                 children: [
41                   Row(
42                     mainAxisAlignment: MainAxisAlignment.spaceEvenly,
43                     children: [
44                       Text(_userModel![index].id.toString()),
45                       Text(_userModel![index].username),
46                     ],
47                   ),
48                   const SizedBox(
49                     height: 20.0,
50                   ),
51                   Row(
52                     mainAxisAlignment: MainAxisAlignment.spaceEvenly,
53                     children: [
54                       Text(_userModel![index].email),
55                       Text(_userModel![index].website),
56                     ],
57                   ),
58                 ],
59             );
60           },
61         ),
62     );
63   }
64 }
65 }

```

Now, in the above code, we created an object of `List<UserModel>` and then in the `initState` we called a method to get the data from our API. In the method, I have added `Future.delayed`, just

so that we can simulate the API time response. This API that we are using is a fake one and hence it's pretty fast. However, real APIs aren't that fast. So it becomes our duty to show a loader until the API response is available. Once the response is available, we rebuild the UI to show the data. Here, I have just shown a few of the data from the model class.

Output



The image shows a smartphone screen with a blue header bar containing the title "REST API Example". Below the header is a table with 10 rows of data. The table has three columns: an index number, a first name, and an email address. The status bar at the top shows the time 4:44, a Wi-Fi signal, and a battery icon with the word "DEBUG" next to it.

Index	First Name	Email
1	Bret	Sincere@april.biz
2	Antonette	hildegard.org
3	Samantha	Shanna@melissa.tv
4	Karianne	anastasia.net
5	Kamren	Nathan@yesenia.net
6	Leopoldo_Corkery	ramiro.info
7	Elwyn.Skiles	Julianne.OConner@kory.org
8	Maxime_Nienow	kale.biz
9	Delphine	Lucio_Hettinger@annie.ca
10	Moriah.Stanton	demarco.info

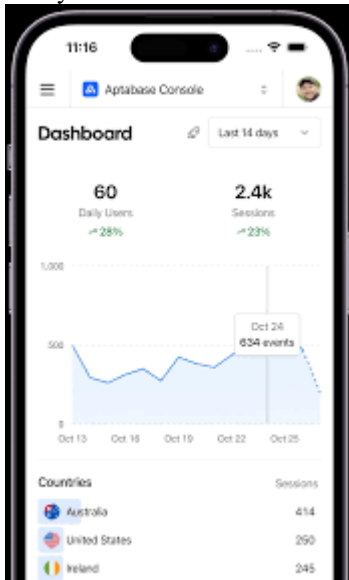
Common APIs that every APP needs nowadays

- API calls for customer authentication.

Subscribe

☒ Remember me for faster sign in

- Analytics



- Backend
- Push Notification APIs (Auth0 etc)
- Integrating Payment Gateways (google pay, paypal) (optional)
- Maps & Location APIs (optional)

main.dart:

Code:

```
import 'package:flutter/material.dart';
import 'screens/home.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Home(),
    );
  }
}
```

pubspec.yaml:

Code:

```
name: mad_lab11
description: "A new Flutter project."
publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ^3.10.7

dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  http: ^0.13.6

dev_dependencies:
  flutter_test:
    sdk: flutter

  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
```

constants.dart:

Code:

```
class ApiConstants {
  static String baseUrl = "https://jsonplaceholder.typicode.com";
  static String usersEndpoint = "/users";
}
```

MODEL CLASS:

user_model.dart:

Code:

```
import 'dart:convert';

List<UserModel> userModelFromJson(String str) =>
  List<UserModel>.from(json.decode(str).map((x) => UserModel.fromJson(x)));

String userModelToJson(List<UserModel> data) =>
  json.encode(List<dynamic>.from(data.map((x) => x.toJson())));
```

```

class UserModel {
    UserModel({
        required this.id,
        required this.name,
        required this.username,
        required this.email,
        required this.address,
        required this.phone,
        required this.website,
        required this.company,
    });

    int id;
    String name;
    String username;
    String email;
    Address address;
    String phone;
    String website;
    Company company;

    factory UserModel.fromJson(Map<String, dynamic> json) => UserModel(
        id: json["id"],
        name: json["name"],
        username: json["username"],
        email: json["email"],
        address: Address.fromJson(json["address"]),
        phone: json["phone"],
        website: json["website"],
        company: Company.fromJson(json["company"]),
    );

    Map<String, dynamic> toJson() => {
        "id": id,
        "name": name,
        "username": username,
        "email": email,
        "address": address.toJson(),
        "phone": phone,
        "website": website,
        "company": company.toJson(),
    };
}

```

```

class Address {
    Address({
        required this.street,
        required this.suite,
        required this.city,
        required this.zipcode,
        required this.geo,
    });

    String street;
    String suite;
    String city;
    String zipcode;
    Geo geo;

    factory Address.fromJson(Map<String, dynamic> json) => Address(
        street: json["street"],
        suite: json["suite"],
        city: json["city"],
        zipcode: json["zipcode"],
        geo: Geo.fromJson(json["geo"]),
    );
}

Map<String, dynamic> toJson() => {
    "street": street,
    "suite": suite,
    "city": city,
    "zipcode": zipcode,
    "geo": geo.toJson(),
};

class Geo {
    Geo({
        required this.lat,
        required this.lng,
    });

    String lat;
    String lng;

    factory Geo.fromJson(Map<String, dynamic> json) => Geo(
        lat: json["lat"],
        lng: json["lng"],
    );
}

```



```

Map<String, dynamic> toJson() => {
  "lat": lat,
  "lng": lng,
};
}

class Company {
  Company({
    required this.name,
    required this.catchPhrase,
    required this.bs,
  });

  String name;
  String catchPhrase;
  String bs;

  factory Company.fromJson(Map<String, dynamic> json) => Company(
    name: json["name"],
    catchPhrase: json["catchPhrase"],
    bs: json["bs"],
  );

  Map<String, dynamic> toJson() => {
    "name": name,
    "catchPhrase": catchPhrase,
    "bs": bs,
  };
}

```

api_service.dart:

Code:

```

import 'dart:developer';
import 'package:http/http.dart' as http;
import 'package:mad_lab11/constants/constants.dart';
import '../model/user_model.dart';

class ApiService {
  Future<List<UserModel>?> getUsers() async {
    try {
      var url = Uri.parse(ApiConstants.baseUrl + ApiConstants.usersEndpoint);
      var response = await http.get(url);

      if (response.statusCode == 200) {
        List<UserModel> model = userModelFromJson(response.body);
        return model;
      }
    }
  }
}

```

```

    }
  } catch (e) {
    log(e.toString());
  }
}
}
}

```

home.dart:

```

import 'package:flutter/material.dart';
import '../model/user_model.dart';
import '../services/api_service.dart';

class Home extends StatefulWidget {
  const Home({Key? key}) : super(key: key);
  @override
  _HomeState createState() => _HomeState();
}

class _HomeState extends State<Home> {
  List<UserModel>? userModel = [];
  @override
  void initState() {
    super.initState();
    getData();
  }

  void getData() async {
    userModel = await ApiService().getUsers();
    setState(() {});
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("REST API Example"),
      ),

      body: userModel == null || userModel!.isEmpty
        ? Center(child: CircularProgressIndicator())
        : ListView.builder(
            itemCount: userModel!.length,
            itemBuilder: (context, index) {

```


Output screen:

localhost:55042

localhost:55042

REST API Example

1	Sincere@april.biz	Bret hildegard.org
2	Shanna@melissa.tv	Antonette anastasia.net
3	Nathan@yesenia.net	Samantha ramiro.info
4	Julianne.OConner@kory.org	Karianne kale.biz
5	Lucio_Hettinger@annie.ca	Kamren demarco.info
6	Karley_Dach@jasper.info	Leopoldo_Corkery ola.org
7	Telly.Hoeger@billy.biz	Elwyn.Skiles elvis.io
8	Sherwood@rosamond.me	Maxime_Nienow jacynthe.com
9	Chaim_McDermott@dana.io	Delphine conrad.com
10	Rey.Padberg@karina.biz	Moriah.Stanton ambrose.net

Actual Output:

REST API Example					
1	Sincere@april.biz	✕	Bret	hildegard.org	
2	Shanna@melissa.tv		Antonette	anastasia.net	
3	Nathan@yesenia.net		Samantha	ramiro.info	
4	Julianne.OConner@kory.org		Karianne	kale.biz	
5	Lucio_Hettinger@annie.ca		Kamren	demarco.info	
6	Karley_Dach@jasper.info		Leopoldo_Corkery	ola.org	
7	Telly.Hoeger@billy.biz		Elwyn.Skiles	elvis.io	
8	Sherwood@rosamond.me		Maxime_Nienow	jacynthe.com	
9	Chaim_McDermott@dana.io		Delphine	conrad.com	
10	Rey.Padberg@karina.biz		Moriah.Stanton	ambrose.net	

Steps:

1. Created a Flutter project in Android Studio.
2. Added http package in pubspec.yaml.
3. Created constants file for API URL.
4. Created model class to parse JSON data.
5. Created API service to fetch data.
6. Displayed API data using widgets like ListView and Card.

Widgets Used:

Scaffold: Used as main UI structure

AppBar: Used to display title

ListView: Used to show list of data

Card: Used to display each user

Row & Column: Used for layout design

Text: Used to show data

CircularProgressIndicator: Used as loading indicator